

Parallel Connected Components Using Hybrid (BFS+SV) vs Parallel DSU

Mukul Nayak(AI25MTECH14008)

1 Objective

The objective of this project is to implement and compare two parallel algorithms for the Connected Components (CC) problem:

- Hybrid Algorithm (BFS + Shiloach-Vishkin)
- Parallel Disjoint Set Union (DSU)

The comparison is performed based on:

- Execution time
- Scalability with graph size
- Effect of graph structure
- Effect of density
- Effect of component distribution
- Parallel speedup

2 Program Design and Approach

2.1 Code Structure

The implementation is modular and consists of:

- Graph module (adjacency list representation)
- Synthetic graph generator
- Hybrid CC implementation
- Parallel DSU implementation
- ParallelFor utility for thread execution
- Experiment driver (compare.cpp)
- Logger for recording results

2.2 Parallelization Strategy

Parallelism is implemented using a custom `parallel_for` construct:

- Work is divided into chunks
- Each thread processes a chunk independently
- Threads are created using `std::thread`
- Synchronization happens via joins at the end

This approach avoids thread pool complexity while still achieving parallel execution.

2.3 Hybrid Algorithm (BFS + SV)

The Hybrid algorithm works in two phases:

2.3.1 Phase 1: Parallel BFS

- Each BFS starts from an unvisited node
- Frontier expansion is parallelized
- Each thread processes part of the frontier
- Threads maintain local buffers to avoid contention

Parallel behavior:

- Threads iterate over frontier nodes
- Each thread explores neighbors independently
- Shared arrays (`visited`, `component`) are updated
- Races are benign (same value written)

2.3.2 Phase 2: Shiloach-Vishkin (SV)

- Operates on remaining nodes
- Uses pointer jumping (compression)
- Uses hooking to merge components

Parallel behavior:

- Each thread processes a subset of edges
- Hooking updates parent pointers
- Compression reduces tree height
- Iterative until convergence

2.4 Parallel DSU (Union-Find)

The DSU algorithm performs:

1. Parallel initialization of parent array
2. Parallel edge processing (union)
3. Path compression
4. Component labeling

Parallel behavior:

- Edges are divided among threads
- Each thread performs union operations
- Atomic CAS ensures correctness
- Find operations use path compression

Important:

- Multiple threads may attempt to union the same nodes
- CAS ensures only one succeeds
- Others retry until stable

3 Experimental Setup

- Nodes: 2000, 5000, 10000
- Graph Types: balanced, mixed_dense, mixed_sparse
- Density: 0.02, 0.05, 0.1
- Thread counts: 1, 2, 4, 8, 12
- Runs per configuration: multiple (averaged)

Graphs are synthetically generated with:

- Dense components
- Medium (scale-free-like) components
- Sparse tree-like components

4 Results and Analysis

4.1 Experiment 1: Runtime vs Graph Size

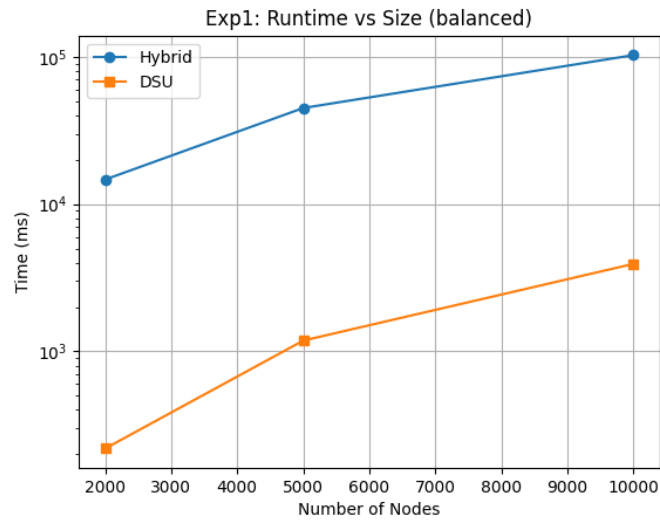


Figure 1: Runtime vs Size (Balanced)

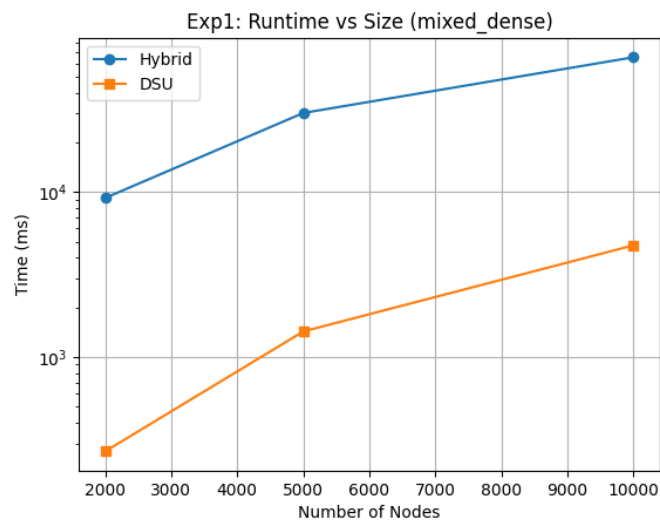


Figure 2: Runtime vs Size (Mixed Dense)

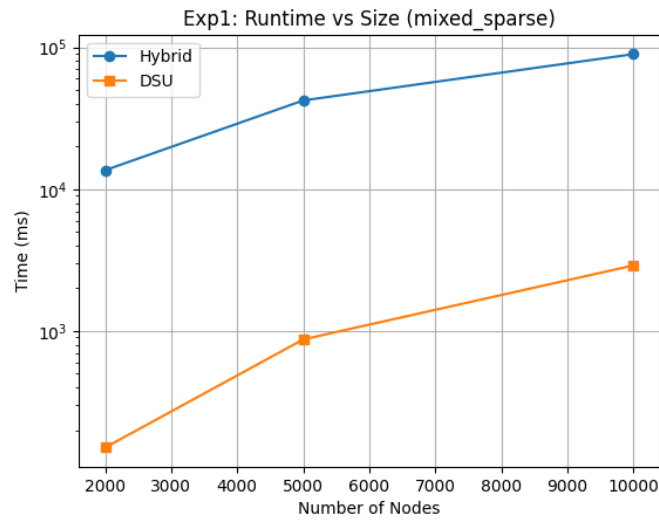


Figure 3: Runtime vs Size (Mixed Sparse)

Observations:

- Runtime increases with graph size
- Hybrid is significantly slower than DSU
- DSU shows near-linear scaling

Reason:

- Hybrid repeatedly scans adjacency lists (high overhead)
- DSU processes edges once (efficient)

4.2 Experiment 2: Runtime vs Graph Type

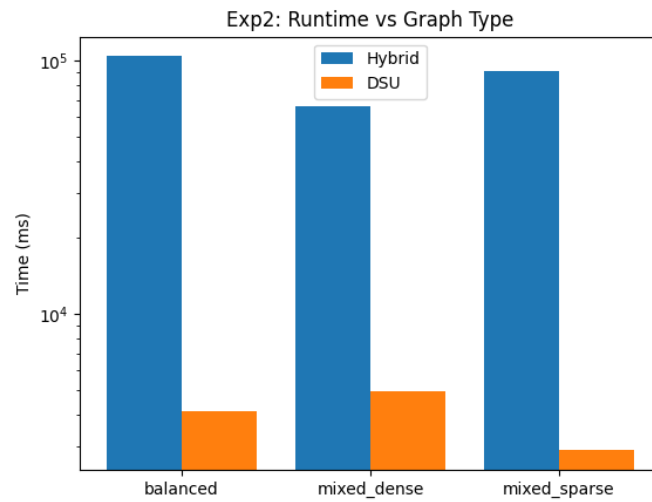


Figure 4: Runtime vs Graph Type

Observations:

- Hybrid varies significantly with graph type
- DSU remains relatively stable

Reason:

- Hybrid depends on structure (BFS behavior)
- DSU depends mainly on edge count

4.3 Experiment 3: Runtime vs Density

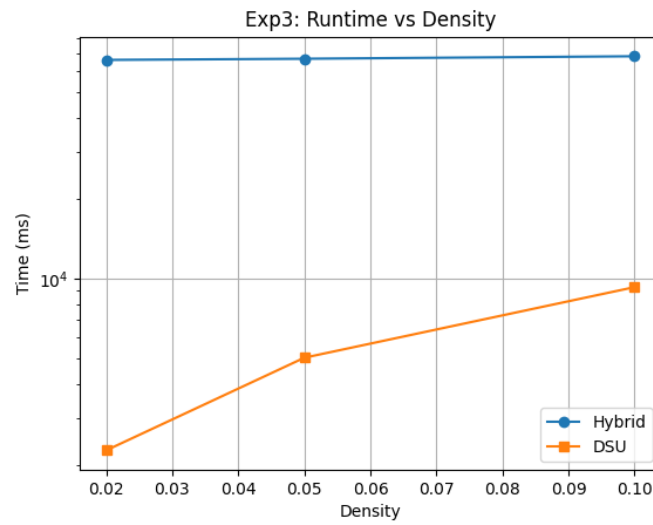


Figure 5: Runtime vs Density

Observations:

- Hybrid runtime remains almost constant
- DSU runtime increases with density

Reason:

- DSU processes all edges (more edges $\rightarrow$ more work)
- Hybrid saturates BFS early

4.4 Experiment 4: Component Distribution

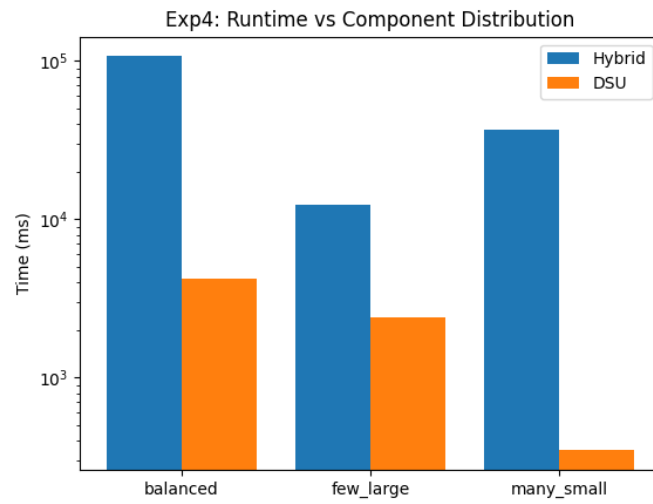


Figure 6: Runtime vs Component Distribution

Observations:

- Hybrid performs best for few large components
- DSU performs best for many small components

4.5 Experiment 5: Speedup vs Threads

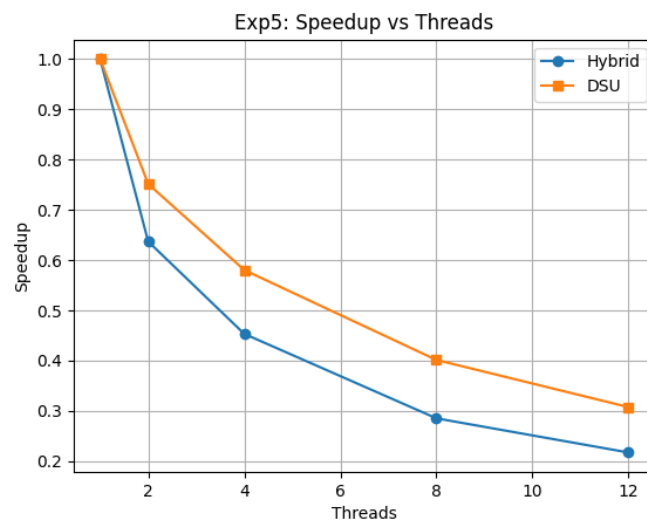


Figure 7: Speedup vs Threads

Observations:

- Both algorithms show poor scaling
- Speedup decreases with more threads

Reason:

- Thread overhead dominates
- Memory contention
- Cache coherence overhead

5 Challenges Faced

- Debugging parallel race conditions
- Avoiding infinite loops in SV
- Correct use of atomic operations
- Managing thread overhead

6 Inspiration from Prior Work

In designing and improving the parallel Disjoint Set Union (DSU) algorithm used in this project, I drew inspiration from two key research works in parallel graph algorithms.

6.1 Provably-Efficient and Internally-Deterministic Parallel Union-Find

I referred to the work by Fedorov et al. [1], which proposes a theoretically efficient and internally deterministic parallel Union-Find algorithm. While the full algorithm involves advanced techniques such as bulk-find and sophisticated batching strategies, several practical insights from this paper influenced its implementation:

- **Deterministic Hooking Strategy:** Instead of arbitrary unions, it ensured consistent parent selection using rank heuristics. This reduces randomness and improves stability under concurrency.
- **Aggressive Path Compression:** The paper emphasizes reducing tree depth aggressively. In our implementation, we apply path compression both during find operations and in a final compression phase to flatten trees.

- **Reducing Contention:** Theoretical analysis in the paper highlights contention as a major bottleneck. I addressed this by minimizing unnecessary retries in union operations and using atomic compare-and-swap (CAS) carefully.
- **Work Efficiency Awareness:** The design ensures that parallel work does not exceed sequential work significantly. Our implementation avoids excessive synchronization and redundant updates.

Although I did not implement bulk-find due to its complexity, the above principles significantly improve the robustness of the parallel DSU.

6.2 Adaptive Parallel Algorithm for Connected Components

I also took inspiration from the work by Jain et al. [2], which proposes an adaptive algorithm for computing connected components using BFS and Shiloach-Vishkin (SV) techniques.

Key insights adopted include:

- **Graph Structure Matters:** The paper shows that different algorithms perform better on different graph topologies. This motivated our experimental design to include multiple graph types (dense, sparse, balanced).
- **Parallel BFS Efficiency:** BFS performs well on low-diameter graphs. Our Hybrid algorithm leverages parallel BFS for large components where traversal is efficient.
- **SV for Small Components:** The SV-style hooking and pointer jumping ideas are effective for handling many small components. Our Hybrid approach incorporates this idea in the second phase.
- **Adaptive Strategy:** Instead of using a single method, the hybrid approach dynamically switches between BFS and SV based on component size.

This paper directly motivates the design of our Hybrid (BFS + SV) algorithm and the need for diverse experimental evaluation.

7 Conclusion

- DSU significantly outperforms Hybrid
- Hybrid is sensitive to graph structure
- Parallelism does not guarantee speedup
- System-level effects dominate performance

The results highlight the gap between theoretical parallel algorithms and practical performance.

References

- [1] Alexander Fedorov, Diba Hashemi, Giorgi Nadiradze, Dan Alistarh, *Provably-Efficient and Internally-Deterministic Parallel Union-Find*, arXiv:2304.09331, 2023.
- [2] Chirag Jain, Patrick Flick, Tony Pan, Oded Green, Srinivas Aluru, *An Adaptive Parallel Algorithm for Computing Connected Components*, IEEE Transactions on Parallel and Distributed Systems, 2017.